Intrinsic Lorentz Attention

Mees Lindeman
Project AI
University of Amsterdam
mees.lindeman@student.uva.nl

Abstract

Hyperbolic geometry provides a natural inductive bias for hierarchical and relational data and has recently been adopted in attention-based models. However, existing hyperbolic attention mechanisms predominantly rely on tangent-space mappings or extrinsic Euclidean transformations followed by projection, introducing geometric approximations that weaken the connection between attention operations and the underlying Riemannian structure. Recent work has proposed intrinsic Lorentzian linear layers that define transformations directly in terms of the Lorentzian metric, but their integration into deeper networks–such as attention mechanisms–remains unexplored. In this work, we propose an attention mechanism constructed from intrinsic Lorentzian linear layers. Focusing on graph-based node classification to isolate attention behavior, we show that intrinsic Lorentzian attention achieves performance comparable to, and in some cases exceeding, existing extrinsic approaches, while preserving geometric consistency. Code available at https://github.com/meeslindeman/project-ai.

1 Introduction

Many real-world datasets exhibit hierarchical or tree-like structure, for which hyperbolic geometry offers a well-matched representational space [3, 12, 25, 26]. Its exponential volume growth allows compact, low-distortion embeddings of such structures, leading to successful applications across representation learning in graphs [4, 7, 14, 23], vision [5, 16], and language [11, 33, 39]. In parallel, attention mechanisms [37] have become a central component of modern deep learning, making the integration of attention and hyperbolic geometry a natural direction.

Most existing hyperbolic attention mechanisms rely on one of two design principles. The first applies Euclidean operations in tangent spaces via exponential and logarithmic maps, while the second performs Euclidean transformations in the ambient space of a hyperbolic model and subsequently enforces manifold constraints through projection or normalization. Although effective in practice, both approaches introduce geometric approximations that weaken the connection between attention operations and the underlying Riemannian structure[1]. Tangent-space methods rely on local linearizations that may accumulate distortion and numerical instability in deep architectures [5, 8, 34] . Projection-based Lorentzian layers, by contrast, treat hyperbolic embeddings as Euclidean features and impose geometry only retrospectively, rather than as part of the learned transformation.

Recent work [34] has proposed an intrinsic Lorentzian linear layer that defines transformations directly in terms of the Lorentzian metric, via a *point-to-hyperplane* construction [20]. This method avoids tangent-space mappings and extrinsic projections, offering a more geometrically principled alternative. However, the suitability as building block for deeper architectures–such as attention mechanism–remains unexplored. This work studies attention mechanisms constructed from such intrinsic Lorentzian linear layers. Rather than introducing a new attention architecture, we adopt a

[1]Riemannian structure refers to the metric tensor that defines distances, angles, and geodesics on the hyperbolic manifold.

unified framework to replace extrinsic Lorentzian transformations with intrinsic ones and analyze the resulting attention block. The focus is on graph-based node classification, which allows attention to be examined without additional architectural components such as positional encoding or normalization layers, whose intrinsic Lorentzian counterparts are not yet well established. Our contributions are summarized as follows:

1. We propose an attention mechanism constructed from intrinsic Lorentzian linear layers, avoiding tangent-space mappings and extrinsic projections within the attention block.
2. We evaluate intrinsic Lorentzian attention on graph-based node classification benchmarks under a unified architectural and optimization framework.
3. We demonstrate that intrinsic Lorentzian attention achieves performance comparable to, and in some cases exceeding, established extrinsic hyperbolic and Euclidean attention baselines.

2 Related work

2.1 Hyperbolic neural networks

Hyperbolic neural networks exploit negatively curved geometry to represent hierarchical and tree-like structures efficiently [3, 12, 25, 26]. The exponential volume growth of hyperbolic space enables low-distortion embeddings of trees and graphs, motivating a broad class of hyperbolic neural architectures. These include hyperbolic fully connected networks [8, 11, 32, 36], convolutional models [5], graph neural networks [7, 23], and attention-based methods [8, 13, 40]. Designing *fully* hyperbolic neural networks requires replacing Euclidean building blocks with operations that are intrinsic to the underlying manifold, rather than relying on mappings or projections to external Euclidean spaces.

Early hyperbolic models predominantly relied on the Poincaré ball [12, 25, 32] due to its conceptual simplicity and closed-form operations. However, later studies identified numerical and optimization challenges associated with this formulation, including gradient instability and sensitivity near the boundary of the manifold [24]. More recent work has shifted toward the Lorentz (hyperboloid) model, which has been shown to improve numerical stability and training robustness [24], leading to stronger performance in vision tasks such as classification and segmentation [5], as well as in graph learning [7] and fully connected architectures [14, 40].

2.2 Hyperbolic attention mechanisms

Attention mechanisms have become a central component of modern deep learning following the success of Transformers [37] and Vision Transformers [9]. Extending attention to hyperbolic space is a natural step when modeling hierarchical or relational data, where Euclidean geometry can be limiting. Early hyperbolic attention methods focused on modifying similarity and aggregation operations. Hyperbolic Attention Networks (HANs) [13] replaced Euclidean dot products with hyperbolic distances and employed the Einstein midpoint [35] for aggregation. Other approaches retained standard Euclidean attention and projected outputs into hyperbolic space for metric learning [10], limiting the role of geometry within the attention mechanism itself.

Later research generalized linear transformations and aggregation operators to the Lorentz manifold. In particular, Chen et al. [8] proposed a fully hyperbolic neural network that employs Lorentzian linear layers and midpoint-based aggregation to define attention-like operations directly on the hyperboloid[2]. More recent work has sought to define attention end-to-end on the Lorentz manifold. HypFormer [40] constructs queries, keys, values, similarity measures, and aggregation operators directly in Lorentz space, and introduces a linearized hyperbolic attention formulation to reduce computational complexity. When restricted to its non-linear attention variant and fixed curvature, HypFormer employs Lorentzian similarity and midpoint-based aggregation closely aligned with earlier fully Lorentzian attention formulations by Chen et al. [8]. Differences relative to prior work arise primarily from architectural choices, such as head fusion and optional curvature scheduling, rather than from fundamentally different geometric primitives.

Despite these advances, most hyperbolic attention mechanisms remain extrinsic in nature, relying on tangent-space mappings or Euclidean transformations followed by projection to enforce manifold

[2]The Lorentzian midpoint is equivalent to the Einstein midpoint under appropriate projection [32].

constraints. As a result, attention operations are not defined intrinsically with respect to the hyperbolic metric. In this work, we adopt an intrinsic Lorentzian perspective and investigate an attention mechanism whose components are defined directly in terms of the Lorentzian metric, avoiding tangent-space approximations and extrinsic projections within the network stack.

3 Background

3.1 Euclidean self-attention

Self-attention was introduced in the Transformer architecture by Vaswani et al. [37] as a mechanism for modeling pairwise interactions within a set of input elements. Unlike recurrent or convolutional operators, self-attention allows each element to directly attend to all others in a single layer, enabling efficient modeling of long-range dependencies. This formulation has since become a foundational building block across a wide range of domains, including natural language processing, vision, and graph representation learning.

Given a set of input features $\mathbf{X} \in \mathbb{R}^{N\times d}$, self-attention constructs three learned representations: queries, keys, and values. These are obtained via linear projections

$$\mathbf{Q} = \mathbf{X}W_q, \qquad \mathbf{K} = \mathbf{X}W_k, \qquad \mathbf{V} = \mathbf{X}W_v,$$

where $W_q, W_k, W_v \in \mathbb{R}^{d\times d}$. The query–key interaction defines a similarity score between elements, while the values encode the information to be aggregated.

In multi-head attention (MHA), the feature dimension is partitioned into H independent heads, each operating on a subspace of dimension d/H. Scaled dot-product attention is computed independently for each head by taking inner products between queries and keys, scaling by $1/\sqrt{d/H}$, and applying softmax normalization to obtain attention weights. These weights are then used to form weighted combinations of the corresponding value vectors. The outputs of all heads are concatenated and linearly projected back to $\mathbb{R}^d$,

$$\mathbf{Y} = \mathrm{MHA}(\mathbf{X})W_o,$$

with $W_o \in \mathbb{R}^{d\times d}$. Geometrically, Euclidean self-attention can be interpreted as computing similarity and aggregation using the standard inner product in $\mathbb{R}^d$.

3.2 Hyperbolic geometry

Hyperbolic geometry [1, 6] is a non-Euclidean geometry characterized by constant negative curvature $\kappa < 0$. Formally, the n-dimensional hyperbolic space $\mathbb{H}^n_\kappa$ is a Riemannian manifold $(\mathcal{M}^n, \mathfrak{g}^\kappa_{\boldsymbol{x}})$ equipped with a Riemannian metric $\mathfrak{g}$ that reflects this curvature. Several isometrically equivalent models represent this geometry, including the Poincaré ball model [11], the Poincaré half-plane model [33], the Klein model [13], and the Lorentz (hyperboloid) model [26]. The Lorentz model is often preferred due to its numerical stability and the simplicity of its closed-form expressions for the distance function, exponential map, and logarithmic map [8, 14, 40].

Lorentz model The Lorentz model represents hyperbolic space as the upper sheet of a two-sheeted hyperboloid embedded in $\mathbb{R}^{n+1}$ equipped with the Lorentzian metric. The manifold $\mathbb{L}^n_\kappa = (\mathbb{L}^n, \mathfrak{g}^\kappa_{\boldsymbol{x}})$ is defined by

$$\mathbb{L}^n := \{\boldsymbol{x} \in \mathbb{R}^{n+1} \mid \langle \boldsymbol{x}, \boldsymbol{x}\rangle_{\mathcal{L}} = \frac{1}{\kappa}, x_t > 0\},$$

where $\kappa < 0$ is constant negative curvature and

$$\langle \boldsymbol{x}, \boldsymbol{x}\rangle_{\mathcal{L}} = -x_t y_t + \boldsymbol{x}_s^\top \boldsymbol{y}_s = \boldsymbol{x}^\top \mathrm{diag}(-1, 1, ..., 1)\boldsymbol{y}.$$

Adapted from the theory of special relativity, the first coordinate x_t is called *time-like* and the remaining n coordinates x_s are *space-like*. Consequently, every point can on the manifold can be expressed as

$$\boldsymbol{x} = [x_t, \boldsymbol{x}_s]^\top, \quad x_t = \sqrt{\|\boldsymbol{x}_s\|^2 - 1/\kappa}.$$

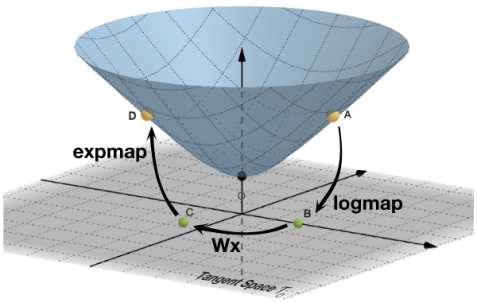


Figure 1: Illustration of Lorentz model and tangent space operations. Reproduced from Chen et al. [8].

The model's origin is $\overline{\mathbf{0}} = [-1/\kappa, 0, ..., 0]^\top$ and distances, geodesics, and tangent spaces are all computed using the Lorentzian inner product $\langle \boldsymbol{x}, \boldsymbol{x} \rangle_{\mathcal{L}}$. The hyperboloid representation leads to numerically stable formulations of hyperbolic geometry and provides closed-form expressions for exponential and logarithmic maps, parallel transport, and distances. Formal definitions are provided in Appendix A.

Tangent spaces A tangent space provides a local linear approximation of a Riemannian manifold at a given point. For the Lorentz manifold $\mathbb{L}^n_\kappa$, the tangent space at x consists of all ambient vectors orthogonal to x under the Lorentzian inner product. Tangent spaces enable the application of Euclidean operations but introduce a local approximation that does not preserve global geometry.

Exponential and logarithmic maps Exponential and logarithmic maps provide smooth transformations between the Lorentz manifold and its tangent spaces. These maps are widely used in hyperbolic neural networks to enable standard linear operations by temporarily mapping representations to a Euclidean space. However, repeated use of exp/log maps introduces additional computational overhead and can lead to numerical instability [5, 8, 34], particularly in deep architectures and near regions of high curvature.

3.3 Lorentz fully connected layers

Many Lorentz-model architectures implement hyperbolic linear layers by applying an unconstrained Euclidean linear transformation to the ambient coordinates of a Lorentz vector, followed by a projection step that enforces the hyperboloid constraint. This design appears both in the hyperbolic linear layer of Chen et al. [8] and in the `HypLinear` module of HypFormer [40]. In these constructions, the learnable transformation itself is Euclidean, while hyperbolic consistency is imposed only through a non-linear normalization.

Formally, let $\boldsymbol{x} \in \mathbb{L}^n_\kappa$ denote a Lorentz vector with time-like component x_0 and spatial component $\boldsymbol{x}_s$. A representative construction applies an ambient Euclidean linear map

$$\phi(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^m,$$

embeds the result into $\mathbb{R}^{m+1}$ by concatenating a zero time coordinate,

$$\boldsymbol{z} = \begin{pmatrix} 0 \\ \phi(\boldsymbol{x}) \end{pmatrix},$$

and subsequently projects back onto the Lorentz manifold by reconstructing the time component,

$$\boldsymbol{y} = \Pi_{\text{time}}(\boldsymbol{z}) = \begin{pmatrix} \sqrt{\|\phi(\boldsymbol{x})\|_2^2 - 1/\kappa} \\ \phi(\boldsymbol{x}) \end{pmatrix}.$$

HypFormer [40] employs a closely related variant. Instead of enforcing a zero time coordinate, it applies a Euclidean linear transformation directly to the ambient Lorentz vector,

$$\boldsymbol{z}_s = W\boldsymbol{x} + \boldsymbol{b},$$

and reconstructs the time-like component using the hyperboloid constraint,

$$z_0 = \sqrt{\|\boldsymbol{z}_s\|_2^2 + \kappa}, \qquad \boldsymbol{z} = [z_0, \boldsymbol{z}_s]^\top.$$

In both cases, the core mapping is Euclidean, and adherence to the Lorentz geometry is enforced only through a subsequent projection step.

As analyzed by Torres [34], this class of constructions exhibits several limitations. First, the resulting mappings are not Lorentz transformations: the function class does not contain the Lorentz group, and additional gating or scaling mechanisms [8] may further restrict outputs to a subset of the hyperboloid. Second, these layers effectively treat hyperbolic embeddings as features processed by a Euclidean network, disregarding the fact that the Riemannian metric on $\mathbb{L}^n_\kappa$ is highly non-uniform. Consequently, identical coordinate perturbations correspond to position-dependent and geometrically inconsistent displacements on the manifold. Finally, time-like and space-like components are processed identically by the Euclidean map, ignoring the intrinsic asymmetry of the Lorentzian metric.

Discriminative learning can be naturally formulated in terms of distances to hyperplanes, where prediction depends on the signed distance of an input to learned decision boundaries rather than on Euclidean affine projections [20]. In hyperbolic space, such distance-based formulations admit a natural geometric interpretation in terms of the manifold's Riemannian metric. Building on this perspective, we rely on the Lorentz fully connected layer introduced by Torres [34], which defines linear transformations intrinsically in terms of hyperbolic geometry. This layer is formulated via signed distances to space-like hyperplanes in $\mathbb{L}^n_\kappa$, yielding a manifold-preserving linear operator. Each of the m output dimensions is parameterized by a Euclidean direction $u^{(j)} \in \mathbb{R}^n$ and a bias $b_j \in \mathbb{R}$, which are reparameterized into a space-like normal vector $v^{(j)} \in \mathbb{R}^{n+1}$,

$$v^{(j)} = \left[\frac{\|u^{(j)}\|_2}{\sinh(\sqrt{\kappa}\, b_j)} \,,\, \cosh(\sqrt{\kappa}\, b_j)\, \frac{u^{(j)}}{\|u^{(j)}\|_2} \right]^\top .$$

Stacking these vectors yields a matrix $V \in \mathbb{R}^{m\times(n+1)}$. Given an input $x \in \mathbb{L}^n_\kappa$, the transformation is defined as

$$z = Vx \in \mathbb{R}^m,$$

which corresponds to computing scaled, signed Lorentzian distances to m hyperplanes. Crucially, this operation is intrinsic: distances are defined with respect to the Lorentzian metric and correspond to geodesic distances between points and hyperplanes on the manifold. A detailed geometric derivation, including the role of geodesics and metric structure, is provided in Torres [34].

A pointwise non-linearity $h(\cdot)$ may subsequently be applied to the resulting distance coordinates. Since non-linear activations do not, in general, preserve the hyperboloid constraint and lack a direct geometric interpretation in hyperbolic space, the time-like component is recomputed solely to enforce membership in $\mathbb{L}^m_\kappa$,

$$z_0 = \sqrt{\frac{1}{\kappa} + \|h(z)\|_2^2}, \qquad \boldsymbol{z} = \begin{bmatrix} z_0 \\ h(z) \end{bmatrix} \in \mathbb{L}^m_\kappa.$$

Importantly, this reconstruction step enforces the manifold constraint but is not part of the linear transformation itself.

Geometrically, this Lorentz fully connected layer operates intrinsically within the Lorentz model: both its parameters (space-like hyperplane normals) and its linear operation (signed distance computation) are defined directly in terms of the Lorentzian metric. Unlike projected Euclidean layers, the learned mapping respects the geometry of $\mathbb{L}^n_\kappa$ by construction. In the remainder of this work, we investigate whether such intrinsic Lorentz linear layers can serve as effective building blocks for attention mechanisms and deeper architectures.

4 Framework

4.1 Common architecture

All models share a unified architectural pipeline. Given a set of N input elements with features $\mathbf{X} \in \mathbb{R}^{N\times d_{\text{in}}}$, an initial Euclidean linear projection maps features to a common hidden dimension, $\mathbf{X} \mapsto \mathbf{H} \in \mathbb{R}^{N\times d}$. Dropout is applied only to these Euclidean input features to avoid introducing stochastic perturbations in non-Euclidean representations. The network core consists of L stacked multi-head attention (MHA) blocks.

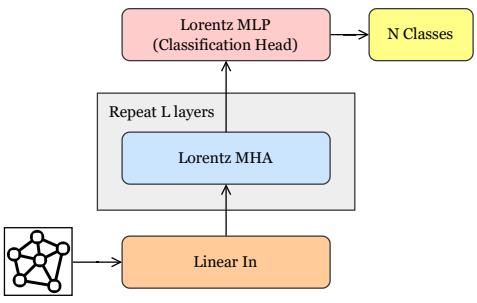


Figure 2: Overview of complete architecture.

All models use the same depth, hidden dimension, and number of attention heads. The only difference between variants lies in the geometric domain in which attention operations and linear transformations are defined. The output of the final MHA block is passed to a task-specific classification head, producing logits $\mathbf{Z} \in \mathbb{R}^{N\times C}$, where C denotes the number of classes.

4.2 Attention variants

We consider four attention mechanisms that share a common high-level block structure but differ in the geometric domain in which linear transformations, similarity computation, and aggregation are defined. This unified formulation enables a controlled comparison across geometric choices while isolating the effect of intrinsic versus extrinsic hyperbolic operations.

Euclidean attention The Euclidean attention block serves as the baseline. All computations are performed in Euclidean space $\mathbb{R}^d$ and follow the standard multi-head attention formulation [37], as introduced in Section 3.1. Queries, keys, and values are obtained via linear projections, attention weights are computed using scaled dot-product similarity, and head outputs are concatenated and projected back to $\mathbb{R}^d$.

Projection Following Ermolov et al. [10], this variant retains standard Euclidean attention throughout the network and maps the final representations into hyperbolic space. In contrast to the original formulation, which targets the Poincaré ball, we employ a Lorentzian classifier layer to embed the Euclidean outputs onto the hyperboloid. This approach enables hyperbolic metric learning at the output level while leaving the attention mechanism itself unchanged.

Extrinsic lorentz attention Extrinsic Lorentz attention refers to attention mechanisms in which embeddings reside on the hyperboloid, but linear transformations are implemented in the ambient Euclidean space and followed by a projection step to enforce the Lorentz constraint. This design is employed by both Chen et al. [8] and HypFormer [40]. Within each attention block, queries, keys, and values are obtained via Lorentz-preserving transformations, similarity is computed using Lorentzian inner products, and aggregation is performed via the Lorentzian midpoint.

When curvature is fixed and the non-linear (quadratic) attention variant is used, HypFormer's attention mechanism is geometrically aligned with the formulation of Chen et al. [8]. In both cases, attention operates on full Lorentz vectors, and aggregation corresponds to geometric averaging on the hyperboloid. Differences between the two approaches arise primarily from architectural choices, such as HypFormer's use of full-dimensional heads and midpoint-based head fusion, rather than from distinct attention geometries.

Intrinsic lorentz attention The proposed attention variant operates intrinsically on the Lorentz manifold. In contrast to extrinsic and tangent-based approaches, attention consumes Lorentz vectors and produces Lorentz vectors at every layer, eliminating repeated exponential and logarithmic maps within the network. Queries, keys, and values are generated using intrinsic Lorentz linear layers [34], and attention scores, aggregation, and head fusion are computed using operators defined directly in terms of the Lorentzian metric. The specific design choices underlying score functions, aggregation rules, and multi-head fusion are detailed in Section 4.3. This formulation realizes a strictly Lorentz-in/Lorentz-out attention operator and enables deep hyperbolic computation without tangent-space approximations or extrinsic projection steps.

4.3 Design choices

This section specifies the design options of the proposed Lorentz attention block. Unless stated otherwise, the reported results use the default settings indicated below.

Score functions Let $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{L}^d_\kappa$ denote Lorentz-values queries and keys. Two score formulations are supported. The default formulation uses the scaled Lorentz (Minkowski) inner product,

$$s = \frac{\langle \boldsymbol{q}, \boldsymbol{k} \rangle_{\mathcal{L}}}{\sqrt{d_{\boldsymbol{h}}}},$$

which serves as the direct analogue of dot-product attention in Euclidean space and is used in all reported experiments.

Alternatively, scores may be derived from the squared Lorentzian (geodesic) distance [19]. For curvature parameter $\kappa < 0$, the squared distance admits the closed form

$$d^2_{\mathcal{L}}(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|^2_{\mathcal{L}} = \frac{2}{\kappa} - 2\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathcal{L}}.$$

In this case, attention scores are obtained from the negative squared distance and scaled by the head dimension.

Value aggregation Value aggregation is performed using the Lorentzian centroid [19]. In the attention setting, the weights are given by the attention coefficients α_{ij} and the vectors are the values v_j,

$$\mathrm{Att}_i \odot_\kappa V = \frac{\sum_{j=1}^{N} \alpha_{ij} v_j}{\sqrt{|\kappa| \left\| \sum_{j=1}^{N} \alpha_{ij} v_j \right\|_{\mathcal{L}}}},$$

where $\odot_\kappa$ represents the weighted sum in hyperbolic space. This operation normalizes the weighted ambient-space sum so that the resulting vector remains on the Lorentz manifold. The midpoint (centroid) formulation is known to provide improved numerical stability compared to exponential-map–based aggregation methods.

Multi-head structure and head fusion We consider two strategies for fusing multi-head attention outputs. With `head_fusion=midpoint`, each head retains full spatial dimensionality, and head outputs are combined using the Lorentzian midpoint operator, as in HypFormer-style fusion. Alternatively, we support Lorentz direct concatenation [30]. Given Lorentz vectors $\{x_i\}_{i=1}^{H}$, where each $x_i \in \mathbb{L}_\kappa^{n_i}$ and $\sum_{i=1}^{H} n_i = n$, their direct concatenation is defined as

$$\mathrm{HCat}(\{x_i\}_i^H = 1) = \left[\sqrt{\sum_{i=1}^{N} x_{i_t}^2 + \frac{N-1}{\kappa}}, \mathbf{x}_{1_s}^\top, ..., \mathbf{x}_{N_s}^\top \right]^\top,$$

where $x_{i,t}$ and $\mathbf{x}_{i,s}$ denote the time-like and spatial components, respectively. This operation can be interpreted as Euclidean concatenation of the spatial components followed by a projection back onto the Lorentz manifold [30].

In addition, we implement a log-radius–scaled concatenation variant inspired by recent unpublished work [2]. In this approach, concatenation is designed to preserve the expected log-radius of Lorentz representations across varying dimensionalities. Since naively stacking spatial components causes the feature norm to grow with the number of concatenated blocks, each block is rescaled such that the expected log spatial radius remains invariant to the total concatenated dimension. This log-radius alignment is parameter-free and aims to improve numerical stability by preventing norm inflation as channel count or kernel size increases [2].

Normalization After the initial Euclidean input projection ℓ_2-normalization is applied to all models for training stability. In the hyperbolic case, this acts as an extrinsic radius control mechanism rather than an intrinsic geometric operation. Layer normalization is omitted inside Lorentz attention blocks due to the absence of a widely accepted and empirically robust normalization scheme for intrinsic Lorentz representations.

5 Experiments

The aim of this work is to study hyperbolic attention mechanisms in relative isolation, minimizing the influence of architectural components whose intrinsically hyperbolic formulations remain underexplored or unavailable. In existing approaches, such components are typically implemented via tangent-space mappings or extrinsic projections, and developing fully intrinsic alternatives is beyond the scope of this study. To reduce these factors, we focus on node classification on graph-structured data, a setting in which attention operates directly on node representations and does not require additional elements such as positional encodings or normalization layers.

Datasets All models are evaluated on five node classification benchmarks commonly used in graph transformer and hyperbolic representation learning literature [7, 14, 38]. `Airport` and `Disease` exhibit a more hierarchical structure [14]. In contrast, `Chameleon`, `Squirrel`, and `Actor` are heterophilous graphs in which neighboring nodes often belong to different classes, providing a complementary evaluation setting. Dataset statistics are summarized in Table 1. Following HGCN [7], for `Disease` and `Airport` we randomly sample train/validation/test splits using fixed ratios

(30/10/60 and 70/15/15, respectively). For `Chameleon`, `Squirrel`, and `Actor`, we use the predefined splits from prior work [22, 29]. All models are trained once per split, and we report mean classification accuracy with sample standard deviation across splits.

Table 1: Datasets statistics [7, 27].

Dataset	AIRPORT	DISEASE	CHAMELEON	SQUIRREL	ACTOR
# Nodes	3188	1044	2277	5201	7600
# Edges	18631	1043	36101	217073	33544
# Features	4	1000	2325	2089	931
# Classes	4	2	5	5	5

Dataset hyperbolicity A graph's hierarchical structure can be characterized using Gromov's δ-hyperbolicity [15]. This quantity measures how tree-like a graph is, with lower values being more hyperbolic [7]. For all datasets, the reported δ values are taken directly from previous work [7, 14] and are used to contextualize model behavior. Additional structural statistics, including core depth and effective diameter, are reported in Appendix C and support the characterization of hierarchical structure across datasets.

Training protocol All models are trained under a unified optimization and evaluation protocol. Euclidean variants are optimized using Adam [17], while hyperbolic variants use RiemannianAdam [18]. Training schedules, early stopping criteria, and architectural hyperparameters are shared across all experiments. Specifically, all models use a hidden dimension of 64, two layers, and two attention heads. Optimization hyperparameters (learning rate, weight decay, and dropout) are selected independently for each dataset and model variant via a small random search over a fixed search space, with an identical trial budget and selection criterion. For each dataset–model pair, the configuration achieving the best validation performance is used for reporting test results.

For hyperbolic models, curvature κ is treated as a tunable parameter, and its sensitivity is analyzed separately in Appendix D. Unless otherwise stated, all main hyperbolic attention experiments employ midpoint-based head fusion, consistent with HypFormer's requirement of preserving full head dimensionality. Variants involving head splitting are evaluated only for our model to isolate the effect of dimensional splitting, with results reported in Section 5.2.

5.1 Main results

Table 2 reports node classification accuracy on both homophilous and heterophilic graph benchmarks. Across all datasets, intrinsic Lorentz attention achieves performance that is competitive with, and in several cases superior to, both Euclidean baselines and extrinsic Lorentz attention mechanisms. This indicates that enforcing geometric consistency through intrinsic Lorentzian operations yields predictive performance comparable to widely used projection-based and tangent-space formulations.

Table 2: Test accuracy (%) for Node Classification on homophilous and heterophilic datasets.

	Homophilous		**Heterophilic**		
Dataset	AIRPORT	DISEASE	CHAMELEON	SQUIRREL	ACTOR
Hyperbolicity	$\delta = 1$	$\delta = 0$	$\delta = 2$	$\delta = 1.5$	$\delta = 1.5$
Euclidean	82.56 ± 1.54	85.59 ± 2.79	38.20 ± 2.51	34.80 ± 1.39	$\mathbf{30.85} \pm 1.08$
Projection	81.20 ± 2.06	85.31 ± 3.33	38.02 ± 2.85	$\mathbf{34.95} \pm 0.91$	28.68 ± 1.20
Chen [8, 40]	87.84 ± 1.65	87.72 ± 3.42	37.94 ± 3.05	34.76 ± 1.01	28.28 ± 1.01
Ours	$\mathbf{90.65} \pm 1.59$	$\mathbf{88.47} \pm 3.31$	$\mathbf{38.53} \pm 2.44$	34.82 ± 1.72	29.63 ± 0.96

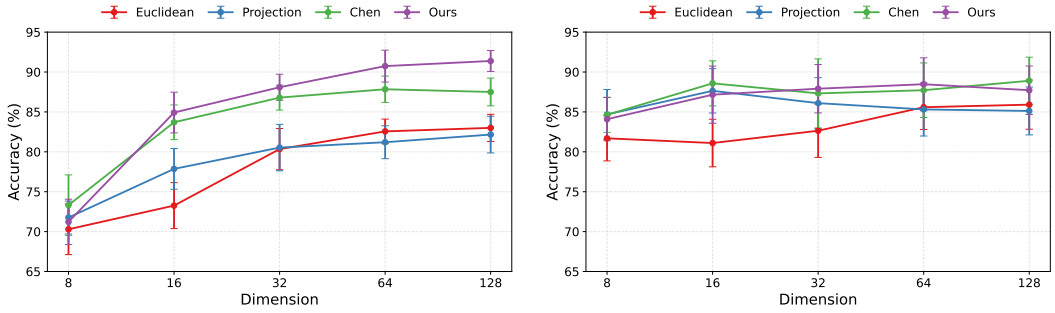
On homophilous datasets (`Airport`, `Disease`), hyperbolic models consistently outperform Euclidean baselines. The intrinsic variant achieves the highest mean accuracy on both benchmarks, suggesting that geometry-aware attention provides a beneficial inductive prior when graph structure aligns with hierarchical organization. The improvements over extrinsic Lorentz attention are moderate but consistent, indicating that intrinsic formulations preserve the advantages of hyperbolic geometry while

enforcing stronger geometric coherence. On heterophilic datasets (`Chameleon`, `Squirrel`, `Actor`), performance differences across methods are smaller, reflecting the increased difficulty of these benchmarks and the reduced role of homophily. Nevertheless, intrinsic Lorentz attention remains competitive with both Euclidean and extrinsic hyperbolic variants. On `Chameleon`, it achieves the highest average accuracy, while on `Squirrel` it closely matches the best-performing baselines.

Performance trends further suggest an interaction between structural signal and feature dimensionality. Datasets with low-dimensional node features and stronger hierarchical structure (e.g., `Airport`) exhibit clearer gains from intrinsic Lorentz attention, whereas datasets with high-dimensional features show smaller differences across models. This observation is consistent with the view that a geometry-aligned inductive prior reduces representational burden when structural information dominates, while its impact diminishes in feature-rich regimes. Overall, the results demonstrate that intrinsic Lorentz attention provides a viable alternative to extrinsic constructions, achieving stable training and competitive accuracy while maintaining geometric consistency. While gains are incremental rather than dramatic, they establish that attention mechanisms can be formulated intrinsically in Lorentzian geometry without sacrificing empirical effectiveness.

5.2 Additional analysis

Embedding dimensionality Hyperbolic neural networks have been shown to be particularly effective in low-dimensional embedding regimes [5, 28]. To assess whether this behavior extends to intrinsic Lorentz attention, we evaluate model performance under progressively reduced embedding dimensions on the `Airport` and `Disease` datasets.



(a) AIRPORT (b) DISEASE

Figure 3: Classification accuracy and standard deviation as a function of embedding dimensionality on the `Airport` and `Disease` datasets.

As shown in Figure 3, performance degrades as dimensionality decreases for all methods; however, intrinsic Lorentz attention exhibits a slower degradation compared to the Euclidean baseline and remains competitive with extrinsic hyperbolic counterparts across dimensions. In particular, at lower embedding sizes, intrinsic attention maintains stable accuracy, indicating that geometric inductive bias mitigates representational bottlenecks when capacity is constrained. These results are consistent with prior observations that hyperbolic geometry supports more compact representations, and suggest that intrinsic formulations preserve this advantage without relying on tangent-space approximations.

Head splitting Table 5 presents an ablation comparing three head-fusion strategies: midpoint fusion, direct Lorentzian concatenation, and log-radius–scaled concatenation (Section 4.3). Across datasets, direct concatenation achieves performance comparable to midpoint fusion, with no systematic degradation despite introducing explicit dimensional splitting across heads. This indicates that manifold-aware concatenation is a viable alternative to midpoint-based fusion, enabling multi-head attention patterns closer to standard Transformer designs without loss of accuracy. The log-radius variant yields slightly higher performance on homophilous datasets (`Airport`, `Disease`) but does not consistently improve results on heterophilous benchmarks, suggesting that its benefit may depend on the strength of hierarchical structure. The compared fusion strategies differ in how representational capacity is allocated. In standard multi-head attention, head splitting reduces per-head dimensionality; accordingly, for concatenation-based variants we increase the hidden dimension to approximately

match the parameter count of midpoint fusion, where heads retain full dimensionality. Overall, these results demonstrate that head splitting with Lorentz-compatible concatenation is feasible and stable, extending the design space of intrinsic Lorentz attention. Concatenation-based strategies facilitate closer alignment with standard Transformer architectures and warrant further investigation in more expressive settings.

Table 3: Comparison of manifold-aware head-fusion strategies in intrinsic Lorentz attention.

Curvature **Hyperbolicity**	AIRPORT $\delta = 1$	DISEASE $\delta = 0$	CHAMELEON $\delta = 2$	SQUIRREL $\delta = 1.5$	ACTOR $\delta = 1.5$
midpoint	90.65 ± 1.59	88.47 ± 3.31	$\mathbf{38.53} \pm 2.44$	$\mathbf{34.82} \pm 1.72$	29.63 ± 0.96
direct	90.17 ± 1.16	88.39 ± 4.07	38.22 ± 2.73	34.33 ± 1.70	$\mathbf{29.99} \pm 0.96$
log-radius	$\mathbf{91.28} \pm 1.94$	$\mathbf{89.29} \pm 3.49$	38.09 ± 2.41	33.41 ± 1.70	29.09 ± 1.24

6 Conclusion

This work studied attention mechanisms formulated intrinsically in Lorentzian geometry, with the goal of assessing whether geometry-consistent operations can function as effective and stable building blocks within attention architectures. Under a unified framework and controlled experimental setting, intrinsic Lorentz attention achieves performance comparable to established Euclidean and extrinsic hyperbolic approaches across a range of graph benchmarks. These results demonstrate that enforcing geometric consistency at the level of linear transformations, similarity computation, and aggregation does not hinder empirical performance. Beyond accuracy, the proposed framework expands the design of hyperbolic attention by enabling multi-head attention, including manifold-aware head splitting and concatenation. While midpoint-based fusion remains a strong default, the feasibility of concatenation-based strategies shows that architectural patterns common in standard Transformers can be realized within a fully Lorentzian setting. Overall, this work establishes intrinsic Lorentz attention as a viable and geometrically grounded alternative to existing hyperbolic attention mechanisms, and provides a principled foundation for the development of more expressive and fully intrinsic hyperbolic architectures.

7 Discussion and limitations

The empirical findings suggest that the benefits of intrinsic Lorentz attention are most pronounced in settings where structural information dominates over rich node features. In particular, datasets with stronger hierarchical characteristics and low-dimensional feature spaces exhibit clearer gains relative to Euclidean baselines, while feature-rich or weakly hierarchical graphs show smaller performance differences. This is consistent with the interpretation that a geometry-aligned inductive prior can reduce representational burden when structural relationships are central to the task.

Several limitations should be noted. First, representations are mapped to the Lorentz manifold through a single Euclidean-to-Lorentz projection at the network input, enabling all subsequent computations to be carried out intrinsically. While this design ensures manifold-valued representations throughout the network, initializing representations directly on the hyperbolic manifold may further improve geometric coherence and is left for future work. Second, the evaluation is restricted to graph-based node classification, chosen to isolate attention behavior without introducing additional components such as positional encoding or normalization layers, whose intrinsic Lorentzian counterparts are not yet well established. As a result, the applicability of intrinsic Lorentz attention to vision or sequence modeling remains an open question. Finally, while head splitting and manifold-aware concatenation are shown to be feasible and stable, the current ablations are not sufficient to draw definitive conclusions about their relative advantages. More expressive architectures may provide a more appropriate setting for separating the effects of parameterization, fusion strategy, and geometric inductive bias. Overall, these considerations highlight that the primary contribution of this work lies in clarifying what is possible within an intrinsic Lorentzian attention framework. Rather than optimizing for maximal performance, the results delineate a principled and extensible design space that can support future work on fully intrinsic hyperbolic deep learning architectures.

References

[1] *Foundations of hyperbolic manifolds*. Springer, 2006.

[2] Anonymous. Intrinsic lorentz neural network. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=NNnkLi1ALt.

[3] Mina Ghadimi Atigh, Julian Schoep, Erman Acar, Nanne Van Noord, and Pascal Mettes. Hyperbolic image segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4453–4462, 2022.

[4] Yushi Bai, Zhitao Ying, Hongyu Ren, and Jure Leskovec. Modeling heterogeneous hierarchies with relation-specific hyperbolic cones. *Advances in Neural Information Processing Systems*, 34:12316–12327, 2021.

[5] Ahmad Bdeir, Kristian Schwethelm, and Niels Landwehr. Fully hyperbolic convolutional neural networks for computer vision. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=ekz1hN5QNh.

[6] James W Cannon, William J Floyd, Richard Kenyon, Walter R Parry, et al. Hyperbolic geometry. *Flavors of geometry*, 31(59-115):2, 1997.

[7] Ines Chami, Zhitao Ying, Christopher Ré, and Jure Leskovec. Hyperbolic graph convolutional neural networks. *Advances in neural information processing systems*, 32, 2019.

[8] Weize Chen, Xu Han, Yankai Lin, Hexu Zhao, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. Fully hyperbolic neural networks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5672–5686, 2022.

[9] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[10] Aleksandr Ermolov, Leyla Mirvakhabova, Valentin Khrulkov, Nicu Sebe, and Ivan Oseledets. Hyperbolic vision transformers: Combining improvements in metric learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7409–7419, 2022.

[11] Octavian Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. *Advances in neural information processing systems*, 31, 2018.

[12] Octavian Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic entailment cones for learning hierarchical embeddings. In *International conference on machine learning*, pages 1646–1655. PMLR, 2018.

[13] Çaglar Gülçehre, Misha Denil, Mateusz Malinowski, Ali Razavi, Razvan Pascanu, Karl Moritz Hermann, Peter W. Battaglia, Victor Bapst, David Raposo, Adam Santoro, and Nando de Freitas. Hyperbolic attention networks. *CoRR*, abs/1805.09786, 2018. URL http://arxiv.org/abs/1805.09786.

[14] Neil He, Menglin Yang, and Rex Ying. Lorentzian residual neural networks, 2025. URL https://arxiv.org/abs/2412.14695.

[15] Edmond Jonckheere, Poonsuk Lohsoonthorn, and Francis Bonahon. Scaled gromov hyperbolic graphs. *Journal of Graph Theory*, 57(2):157–180, 2008.

[16] Valentin Khrulkov, Leyla Mirvakhabova, Evgeniya Ustinova, Ivan Oseledets, and Victor Lempitsky. Hyperbolic image embeddings. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6418–6428, 2020.

[17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.

[18] Max Kochurov, Rasul Karimov, and Serge Kozlukov. Geoopt: Riemannian optimization in pytorch, 2020.

[19] Marc Law, Renjie Liao, Jake Snell, and Richard Zemel. Lorentzian distance learning for hyperbolic representations. In *International Conference on Machine Learning*, pages 3672–3681. PMLR, 2019.

[20] Guy Lebanon and John Lafferty. Hyperplane margin classifiers on the multinomial manifold. In *Proceedings of the twenty-first international conference on Machine learning*, page 66, 2004.

[21] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es, 2007.

[22] Derek Lim, Xiuyu Li, Felix Hohne, and Ser-Nam Lim. New benchmarks for learning on non-homophilous graphs. *arXiv preprint arXiv:2104.01404*, 2021.

[23] Qi Liu, Maximilian Nickel, and Douwe Kiela. Hyperbolic graph neural networks. *Advances in neural information processing systems*, 32, 2019.

[24] Gal Mishne, Zhengchao Wan, Yusu Wang, and Sheng Yang. The numerical stability of hyperbolic representation learning. In *International Conference on Machine Learning*, pages 24925–24949. PMLR, 2023.

[25] Maximillian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. *Advances in neural information processing systems*, 30, 2017.

[26] Maximillian Nickel and Douwe Kiela. Learning continuous hierarchies in the lorentz model of hyperbolic geometry. In *International conference on machine learning*, pages 3779–3788. PMLR, 2018.

[27] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287*, 2020.

[28] Wei Peng, Tuomas Varanka, Abdelrahman Mostafa, Henglin Shi, and Guoying Zhao. Hyperbolic deep neural networks: A survey. *IEEE Transactions on pattern analysis and machine intelligence*, 44(12):10023–10044, 2021.

[29] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A critical look at the evaluation of gnns under heterophily: Are we really making progress? *arXiv preprint arXiv:2302.11640*, 2023.

[30] Eric Qu and Dongmian Zou. Autoencoding hyperbolic representation for adversarial generation. *arXiv preprint arXiv:2201.12825*, 2022.

[31] Stephen B Seidman. Network structure and minimum degree. *Social networks*, 5(3):269–287, 1983.

[32] Ryohei Shimizu, Yusuke Mukuta, and Tatsuya Harada. Hyperbolic neural networks++. *arXiv preprint arXiv:2006.08210*, 2020.

[33] Alexandru Tifrea, Gary Bécigneul, and Octavian-Eugen Ganea. Poincar\'e glove: Hyperbolic word embeddings. *arXiv preprint arXiv:1810.06546*, 2018.

[34] Ricardo Chávez Torres. Lorentzian neural networks++. Unpublished thesis, 2025.

[35] Abraham Albert Ungar. *Analytic hyperbolic geometry: Mathematical foundations and applications*. World Scientific, 2005.

[36] Max Van Spengler, Erwin Berkhout, and Pascal Mettes. Poincare resnet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5419–5428, 2023.

[37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL `http://arxiv.org/abs/1706.03762`.

[38] Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. Sgformer: Simplifying and empowering transformers for large-graph representations. *Advances in Neural Information Processing Systems*, 36:64753–64773, 2023.

[39] Menglin Yang, Aosong Feng, Bo Xiong, Jihong Liu, Irwin King, and Rex Ying. Hyperbolic fine-tuning for large language models. *arXiv preprint arXiv:2410.04010*, 2024.

[40] Menglin Yang, Harshit Verma, Delvin Ce Zhang, Jiahong Liu, Irwin King, and Rex Ying. Hypformer: Exploring efficient hyperbolic transformer fully in hyperbolic space. *arXiv preprint arXiv:2407.01290*, 2024.

A Manifold operations

Distance Distance is defined as the length of the shortest path between two points on a surface. In Euclidean geometry, this path is a straight line. In hyperbolic space, however, the shortest path is a curved geodesic that generalizes the concept of a straight line. In the Lorentz model, distances are derived from the Minkowski metric of the ambient space. Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{L}_\kappa^n$ denote two points in the Lorentz model. Then, the length of the connecting geodesic, and thus distance, is given by

$$d_{\mathcal{L}}(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{-\kappa}} \text{arccosh}(\kappa \langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathcal{L}}.$$

When calculating the distance of any point $\boldsymbol{x} \in \mathbb{L}_\kappa^n$ to the origin $\overline{\mathbf{0}}$, this simplifies to

$$d_{\mathcal{L}}(\boldsymbol{x}, \overline{\mathbf{0}}) = \| \log_{\overline{\mathbf{0}}}^{\kappa}(\boldsymbol{x}) \|.$$

Tangent space A tangent space provides a linear, first-order approximation of a differentiable manifold at a given point. For each point $\boldsymbol{x} \in \mathcal{M}$, the tangent space $T_{\boldsymbol{x}}\mathcal{M}$ is a vector space consisting of all possible directional derivatives passing through $\boldsymbol{x}$. This linearization enables Euclidean operations but also introduces approximation errors. For the Lorentzian manifold $\mathbb{L}_\kappa^n$, the tangent space at $\boldsymbol{x}$ is expressed as

$$T_{\boldsymbol{x}}\mathbb{L}_\kappa^n := \{ \boldsymbol{y} \in \mathbb{R}^{n+1} \mid \langle \boldsymbol{y}, \boldsymbol{x} \rangle_{\mathcal{L}} = 0 \},$$

i.e., all vectors in ambient Minkowski space orthogonal to x under the Lorentzian inner product.

Exponential and logarithmic maps Exponential and logarithmic maps provide smooth transformations between a manifold and its tangent spaces. For a point $\boldsymbol{x} \in \mathbb{L}_K^n$, the exponential map $\exp_{\boldsymbol{x}}^K : T_{\boldsymbol{x}}\mathbb{L}_\kappa^n \to \mathbb{L}_\kappa^n$ maps a tangent vector $\boldsymbol{z} \in T_{\boldsymbol{x}}\mathbb{L}_\kappa^n$ on the Lorentz manifold by

$$\exp_{\boldsymbol{x}}^{\kappa}(\boldsymbol{z}) = \cosh(\alpha)\boldsymbol{x} + \sinh(\alpha)\frac{\boldsymbol{z}}{\alpha}, \;\; \alpha = \sqrt{-\kappa}||\boldsymbol{z}||_{\mathcal{L}}, \;\; ||\boldsymbol{z}||_{\mathcal{L}} = \sqrt{\langle \boldsymbol{z}, \boldsymbol{z} \rangle_{\mathcal{L}}}.$$

When the reference point is the origin, the map simplifies to

$$\exp_{\overline{\mathbf{0}}}^{\kappa}(\boldsymbol{z}) = \frac{1}{\sqrt{-\kappa}} \left[\cosh(\sqrt{-\kappa}||\boldsymbol{z}||), \;\; \sinh(\sqrt{-\kappa}||\boldsymbol{z}||) \frac{\boldsymbol{z}}{||\boldsymbol{z}||} \right].$$

The logarithmic map $\log_{\boldsymbol{x}}^{\kappa} : \mathbb{L}_\kappa^n \to T_{\boldsymbol{x}}\mathbb{L}_\kappa^n$ is the inverse operation and maps a point $\boldsymbol{y} \in \mathbb{L}_\kappa^n$ to the tangent space of $\boldsymbol{x}$ by

$$\log_{\boldsymbol{x}}^{\kappa}(\boldsymbol{y}) = \frac{\text{arccosh}(\beta)}{\sqrt{\beta^2 - 1}} \cdot (\boldsymbol{y} - \boldsymbol{\beta}\boldsymbol{x}), \;\; \beta = \kappa \langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathcal{L}}.$$

Where $\log_{\boldsymbol{x}}^{\kappa}(\exp_{\boldsymbol{x}}^{\kappa}(\boldsymbol{z})) = \boldsymbol{z}$. Together, exponential and logarithmic maps provide the fundamental tools for moving between nonlinear hyperbolic geometry and its linear tangent approximations.

B Implementation details

All models are trained for a maximum of 500 epochs with early stopping triggered after 400 epochs without validation improvement. Training is conducted in a single-graph setting with batch size 1. Architectural parameters are fixed across all experiments, including a hidden dimension of 64, two layers, two attention heads, and adjacency-based attention scope.

Optimization hyperparameters are selected via a small random hyperparameter sweep performed independently for each dataset and model variant. The sweep explores learning rates $\{1 \times 10^{-2}, 1 \times 10^{-3}, 3 \times 10^{-3}, 5 \times 10^{-3}\}$, weight decay values $\{1 \times 10^{-4}, 1 \times 10^{-3}, 3 \times 10^{-3}\}$, and dropout rates $\{0.0, 0.1, 0.2, 0.3\}$. For hyperbolic models, curvature is fixed at $\kappa = -1.0$ during this sweep. All architectural parameters remain fixed throughout. Final reported results correspond to the best-performing configuration on the validation split for each dataset–model pair.

C Dataset details

To better understand when hyperbolic geometry is beneficial, we characterize the structural properties of each dataset using standard graph-theoretic measures of hierarchy and tree-likeness. Following prior work on graph learning, we report: (i) average degree and leaf fraction to quantify sparsity and branching structure, (ii) k-core number [31] to measure core depth, (iii) effective diameter [21] to capture global graph depth, and (iv) mean BFS depth as an interpretable proxy for hierarchical levels. Table 4 summarizes these statistics for all datasets.

Table 4: Structural and hierarchy statistics of the evaluated datasets.

Dataset	Nodes	Edges	δ	AvgDeg	Leaf%	$\text{Core}_{\max}$	EffDiam	$\text{Depth}_{\text{mean}}$
AIRPORT	3188	18630	1	11.69	22.4	31	5	3.13
DISEASE	1044	1043	0	2.00	75.0	1	10	6.59
CHAMELEON	890	8854	2	19.90	3.8	62	6	3.57
SQUIRREL	2223	46998	1.5	42.28	3.1	152	5	2.74
ACTOR	7600	26659	1.5	7.02	21.4	10	5	3.46

Lower average degree, higher leaf fraction, smaller k-core, larger diameters, and deeper BFS levels indicate more tree-like or hierarchical structure, where hyperbolic representations are expected to incur lower distortion. Conversely, dense graphs with large cores and shallow depths exhibit weaker hierarchy. From this, we can conclude `Disease` is the most tree-like dataset, with average degree 2, 75% leaves, $\text{Core}_{\max} = 1$, and the largest diameter and mean depth, indicating a strong hierarchy where hyperbolic representations should be most beneficial. `Airport` and `Actor` show moderate sparsity and depth, suggesting partial hierarchy. In contrast, `Chameleon` and especially `Squirrel` are dense, core-heavy, and shallow, making them the least tree-like and less naturally suited to hyperbolic geometry. These trends are consistent with the reported δ-hyperbolicity values: lower δ (e.g., `Disease`, `Airport`) aligns with stronger tree-likeness, while higher δ (`Chameleon`, `Squirrel`, `Actor`) indicates greater deviation from an underlying hyperbolic structure.

D Analysis of curvature

In the Lorentz model, the curvature parameter $\kappa < 0$ controls the global geometry of the hyperbolic space, determining the rate at which distances grow and volumes expand away from the origin. Larger magnitudes of curvature (i.e., more negative κ) induce stronger hyperbolicity, leading to increased separation of points and sharper representations of hierarchical structure, while curvature values closer to zero approach the Euclidean limit.

We examine the sensitivity of our model to the choice of curvature κ, running the experiments detailed in Section 5 for $\kappa \in \{-0.5, -1.0, -1.5, -2.0, \text{trainable}\}$ on all datasets. The results are shown in Table 5. We observe no particular sensitivity to curvature.

Table 5: Curvature ablation across all datasets.

Curvature Hyperbolicity	AIRPORT $\delta = 1$	DISEASE $\delta = 0$	CHAMELEON $\delta = 2$	SQUIRREL $\delta = 1.5$	ACTOR $\delta = 1.5$
-0.5	**91.51** $\pm$ 1.28	87.28 $\pm$ 3.63	**38.54** $\pm$ 3.24	34.57 $\pm$ 1.84	29.42 $\pm$ 1.18
-1.0	90.65 $\pm$ 1.59	88.47 $\pm$ 3.14	**38.53** $\pm$ 2.44	34.82 $\pm$ 1.72	29.63 $\pm$ 0.96
-1.5	89.73 $\pm$ 2.33	89.25 $\pm$ 3.19	36.82 $\pm$ 3.21	34.77 $\pm$ 1.43	29.57 $\pm$ 1.33
-2.0	89.87 $\pm$ 1.29	**89.72** $\pm$ 2.61	37.56 $\pm$ 3.09	34.54 $\pm$ 1.26	**29.68** $\pm$ 1.10
Trainable	90.86 $\pm$ 1.46	87.24 $\pm$ 3.14	38.44 $\pm$ 3.69	**34.94** $\pm$ 1.84	29.50 $\pm$ 1.51